

WORKBOOK 3: ADVANCED SIGNAL LOGIC

1. The Constexpr PSD Normalizer

Concepts: constexpr, template specialization, lambdas

Write a `constexpr` function that computes the "Max PSD" of a static array of frequencies at compile-time. Use **template specialization** to handle a specific "Calibration" type that applies a different offset. The final result should allow you to define a `constexpr double norm = calculateNorm<Mode>(data);`.

2. Variadic Band-Pass Filter

Concepts: Fold expressions, arg packs, requires

Create a function `checkBands` that takes a `lowCut` and `highCut` frequency. It then accepts an **argument pack** of any number of frequencies. Use a **fold expression** to return `true` only if ALL frequencies are within the cut-off range. Use **requires** to ensure all inputs are numeric.

3. The Function Pointer Dispatcher

Concepts: Function pointers, passing functions as arguments, lambdas

Write a system where you pass a `double* buffer` and a "processing function".

Implement three different processors using **Lambdas**:

- One that subtracts the max value.
- One that applies a 1/3 octave smoothing (simulated logic).
- One that converts values to Decibels.

Pass these lambdas into a single `executePipe` function using **function pointers** or auto parameters.

4. The Recursive Template Integrator

Concepts: Fold expressions, `constexpr`, arg packs

Create a **fold expression** that doesn't just sum values, but performs a cumulative operation (like a moving average simulation) across a variadic pack of signal samples, returning the final integrated value at compile-time using `constexpr`.